

反射

鸣谢：黑马程序员



一、概述

Module `java.base` 模块 `java.base`

Package `java.lang.reflect`

包 `java.lang.reflect`

package `java.lang.reflect`

Provides classes and interfaces for obtaining reflective information about classes and objects. Reflection allows programmatic access to information about the fields, methods, and constructors of loaded classes, and the use of reflected fields, methods, and constructors to operate on their underlying counterparts, within encapsulation and security restrictions.

提供类和接口，用于获取关于类和对象的反射信息。反射允许程序化访问已加载类的字段、方法和构造函数的信息，并使用反射的字段、方法和构造函数在封装和安全限制内操作它们的底层对应物。

Classes in this package, along with `java.lang.Class` accommodate applications such as debuggers, interpreters, object inspectors, class browsers, and services such as Object Serialization and JavaBeans that need access to either the public members of a target object (based on its runtime class) or the members declared by a given class.

本包中的类以及 `java.lang.Class` 支持诸如调试器、解释器、对象检查器、类浏览器等应用程序，以及需要访问目标对象（基于其运行时类）的公共成员或给定类声明的成员的服务，例如对象序列化和 JavaBeans。

二、操作步骤

1. 加载类，获取类的字节码： `Class` 对象

1.1 获取 `Class` 对象的3种方式

- `Class c1 = 目标类名.class`
- 调用 `Class` 类提供的静态方法: `public static Class.forName(String package)`
 - `Class c2 = Class.forName(目标类的完整类名, 即包括所在包名)`
- 调用 `Object` 类提供的方法: `public Class getClass()`
 - `Class c3 = 目标类的实例对象.getClass()`

1.2 代码演示

首先在模块下新建一个包 `reflection` , 然后创建一个学生类 `Student` 和一个测试类 `Test1` 。

- `Student`:

```
1 package reflection;
2
3 import java.util.Objects;
4
5 public class Student {
6     private String name;
7     private int age;
8     private double height;
9
10    public Student() {
11    }
12
13    public Student(String name, int age, double height) {
14        this.name = name;
15        this.age = age;
16        this.height = height;
17    }
18
19    // 省略getter&setter
20 }
```

- `Test1`:

```
1 package reflection;
2
3 public class Test1 {
```

```

4      public static void main(String[] args) throws Exception {
5          Class c1 = Student.class;
6          System.out.println(c1.getName()); // 完整类名，包括前面的包名
7          System.out.println(c1.getSimpleName()); // 简名: Student
8
9          Class c2 = Class.forName("reflection.Student");
10         if (c2.equals(c1)) {
11             System.out.println("c2 == c1");
12         }
13
14         Student s = new Student();
15         Class c3 = s.getClass();
16         if (c3.equals(c1)) {
17             System.out.println("c3 == c1");
18         }
19     }
20 }

```

- 控制台输出:

```

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-!
reflection.Student
Student
c2 == c1
c3 == c1

Process finished with exit code 0

```

2. 获取类的构造器: `Constructor` 对象

2.1 常用方法

P1 `Class` 类提供的方法

序号	方法	说明
01	<code>Constructor<?>[] getConstructors()</code>	获取全部 <code>public</code> 构造器。

序号	方法	说明
02	<code>Constructor<?>[] getDeclaredConstructors()</code>	获取全部构造器。
03	<code>Constructor<?>[] getConstructor(Class<?>... parameterTypes)</code>	获取某个 <code>public</code> 构造器。
04	<code>Constructor<?>[] getDeclaredConstructor(Class<?>... parameterTypes)</code>	获取某个构造器。

P2 `Constructor` 类提供的方法

序号	方法	说明
01	<code>Object newInstance(Object... initargs)</code>	调用此构造器对象表示的构造器，传入参数，完成对象的初始化并返回。
02	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即 暴力反射 。

2.2 代码演示

在包 `reflection` 下继续创建一个猫类 `Cat` 和一个测试类 `Test2`。

- `Cat`：

```

1 package reflection;
2
3 public class Cat {
4     private String name;
5     private int age;
6
7     public Cat() {
8         System.out.println("no-param constructor is executed");
9     }
10 }

```

```

9      }
10
11     private Cat(String name) {
12         this.name = name;
13         System.out.println("private Cat(String name) is executed");
14     }
15
16     private Cat(int age) {
17         System.out.println("private Cat(int age) is executed");
18         this.age = age;
19     }
20
21     public Cat(String name, int age) {
22         this.name = name;
23         this.age = age;
24         System.out.println("full-params constructor is executed");
25     }
26
27     @Override
28     public String toString() {
29         return "Cat{" +
30             "name='" + name + '\'' +
31             ", age=" + age +
32             '}';
33     }
34
35     // 省略getter&setter
36 }

```

- **Test2:**

```

1  package reflection;
2
3  import java.lang.reflect.Constructor;
4  import java.util.Arrays;
5
6  public class Test2 {
7      public static void main(String[] args) throws Exception {
8          Class c = Cat.class;
9
10         System.out.println("====All Public Constructors====");
11         Arrays.stream(c.getConstructors()).forEach(constructor -> {

```

```

12         System.out.println(constructor.toString() + " ==> " +
    constructor.getParameterCount() + " params");
13     });
14     System.out.println();
15
16     System.out.println("====All Constructors====");
17
18     Arrays.stream(c.getDeclaredConstructors()).forEach(constructor ->
    {
19         System.out.println(constructor.toString() + " ==> " +
    constructor.getParameterCount() + " params");
20     });
21     System.out.println();
22
23     Constructor noParamConstructor = c.getConstructor();
24     System.out.println("no-param constructor: " +
    noParamConstructor);
25
26     Constructor fullParamsConstructor =
    c.getDeclaredConstructor(String.class, int.class);
27     System.out.println("full-params constructor: " +
    fullParamsConstructor);
28 }
29 }

```

- 控制台输出:

```

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\IntelliJ IDEA Ult
====All Public Constructors====
public reflection.Cat() ==> 0 params
public reflection.Cat(java.lang.String,int) ==> 2 params

====All Constructors====
private reflection.Cat(java.lang.String) ==> 1 params
public reflection.Cat() ==> 0 params
private reflection.Cat(int) ==> 1 params
public reflection.Cat(java.lang.String,int) ==> 2 params

no-param constructor: public reflection.Cat()
full-params constructor: public reflection.Cat(java.lang.String,int)

Process finished with exit code 0

```


序号	方法	说明
01	<code>Field[] getFields()</code>	获取全部 <code>public</code> 成员变量。
02	<code>Field[] getDeclaredFields()</code>	获取全部成员变量。
03	<code>Field getField(String name)</code>	获取某个 <code>public</code> 成员变量。
04	<code>Field getDeclaredField(String name)</code>	获取某个成员变量。

P2 `Field` 类提供的方法

序号	方法	说明
01	<code>void set(Object obj, Object value)</code>	赋值。
02	<code>Object get(Object obj)</code>	取值。
03	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即 暴力反射 。

3.2 代码演示

- `Cat` :

```
1 package reflection;
2
3 public class Cat {
4     public static int a;
5     public static final String COUNTRY = "CHINA";
6     private String name;
7     private int age;
8
9     public Cat() {
```



```

10         System.out.println("no-param constructor is executed");
11     }
12
13     private Cat(String name) {
14         this.name = name;
15         System.out.println("private Cat(String name) is executed");
16     }
17
18     private Cat(int age) {
19         System.out.println("private Cat(int age) is executed");
20         this.age = age;
21     }
22
23     public Cat(String name, int age) {
24         this.name = name;
25         this.age = age;
26         System.out.println("full-params constructor is executed");
27     }
28
29     @Override
30     public String toString() {
31         return "Cat{" +
32             "name='" + name + '\'' +
33             ", age=" + age +
34             '\'';
35     }
36
37     // 省略getter&setter
38 }

```

- **Test3:**

```

1  package reflection;
2
3  import java.lang.reflect.Field;
4  import java.util.Arrays;
5
6  public class Test3 {
7      public static void main(String[] args) throws Exception {
8          Class c = Cat.class;
9
10         System.out.println("=====All Fields=====");
11         Arrays.stream(c.getDeclaredFields()).forEach(f -> {

```

```

12         System.out.println(f.getName() + " ==> " +
    f.getType());
13     });
14     System.out.println();
15
16     Field fName = c.getDeclaredField("name");
17     System.out.println(fName.getName() + " ==> " +
    fName.getType());
18 }
19 }

```

- 控制台输出：

```

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\C
=====All Fields=====
a ==> int
COUNTRY ==> class java.lang.String
name ==> class java.lang.String
age ==> int

name ==> class java.lang.String

Process finished with exit code 0

```

3.3 获取成员变量的作用：赋值与取值

```

1 package reflection;
2
3 import java.lang.reflect.Field;
4
5 public class Test3 {
6     public static void main(String[] args) throws Exception {
7         Class c = Cat.class;
8
9         Field fName = c.getDeclaredField("name");
10        System.out.println(fName.getName() + " ==> " +
    fName.getType());
11        fName.setAccessible(true);
12        Cat cat = new Cat();

```

```
13
14         fName.set(cat, "三花猫");
15         System.out.println("赋值后: " + cat);
16         String name = (String) fName.get(cat); // 强转
17         System.out.println("取值: " + name);
18
19     }
20 }
```

控制台输出:

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwa
name ==> class java.lang.String
no-param constructor is executed
赋值后: Cat{name='三花猫', age=0}
取值: 三花猫

Process finished with exit code 0
```

4. 获取类的成员方法: `Method` 对象

4.1 常用方法

P1 `Class` 类提供的方法

P2 Method 类提供的方法

序号	方法	说明
01		
02		
03	<code>void setAccessible(boolean flag)</code>	当 <code>flag</code> 设置为 <code>true</code> 时，表示禁止编译器检查访问权限，即 暴力反射 。

4.2 代码演示

4.3 获取成员方法的作用：

三、作用

